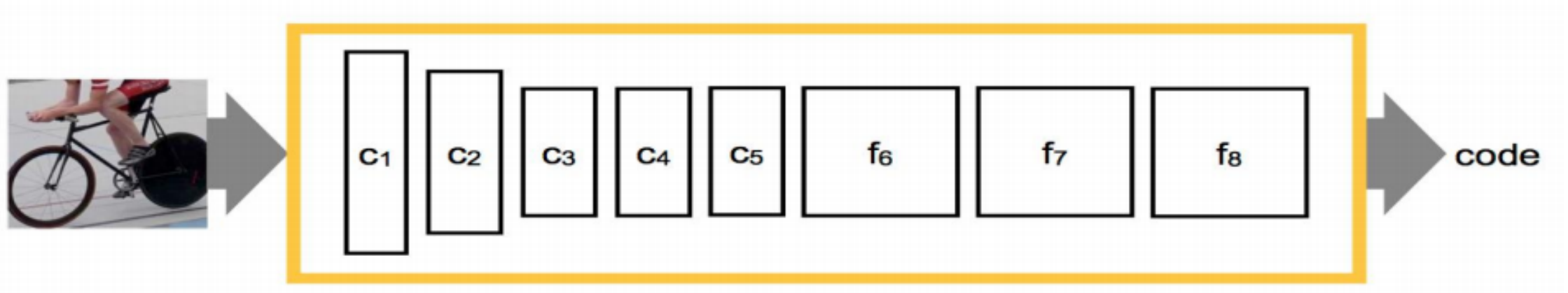


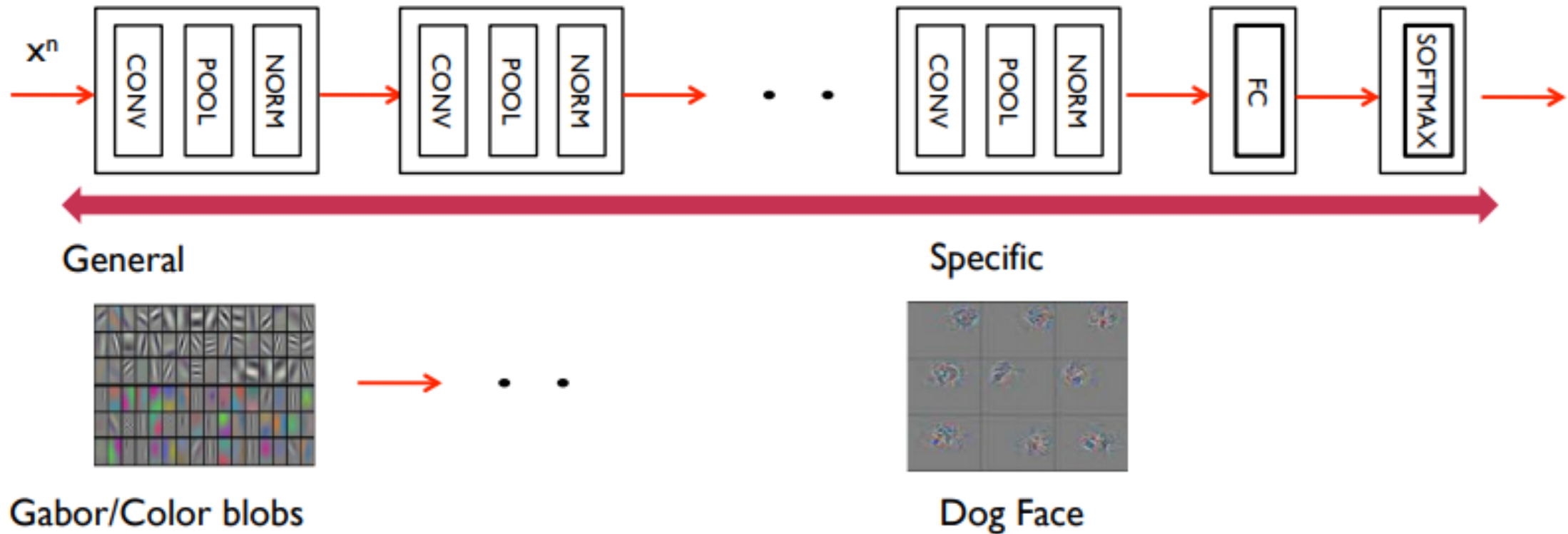
CNN Architectures

Learned Representations: Recap



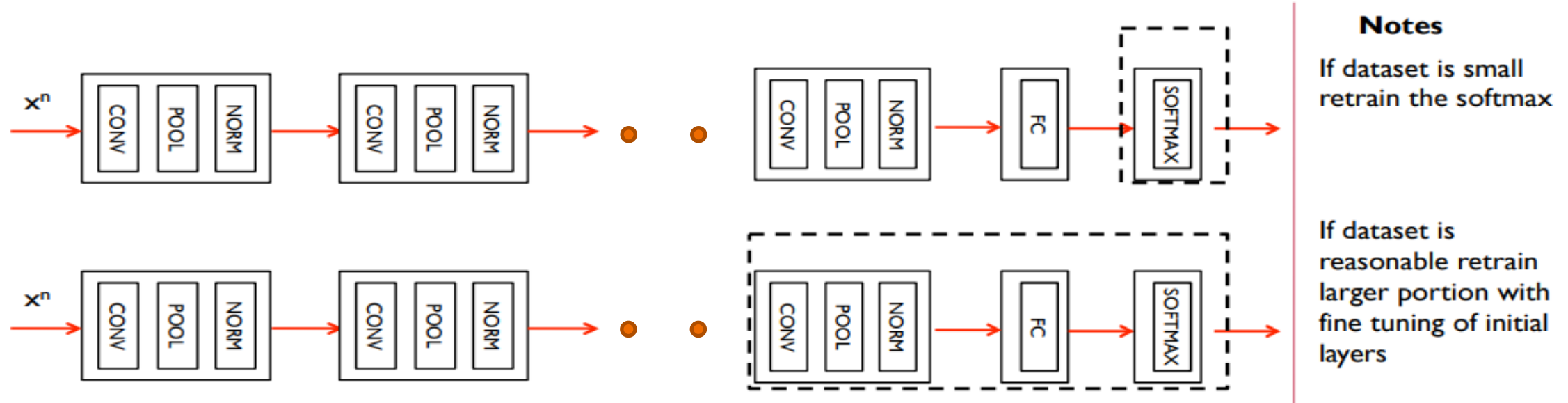
Transfer Learning:

- A key observation that we noticed in visualization:-



Transfer Learning:

- Take away message



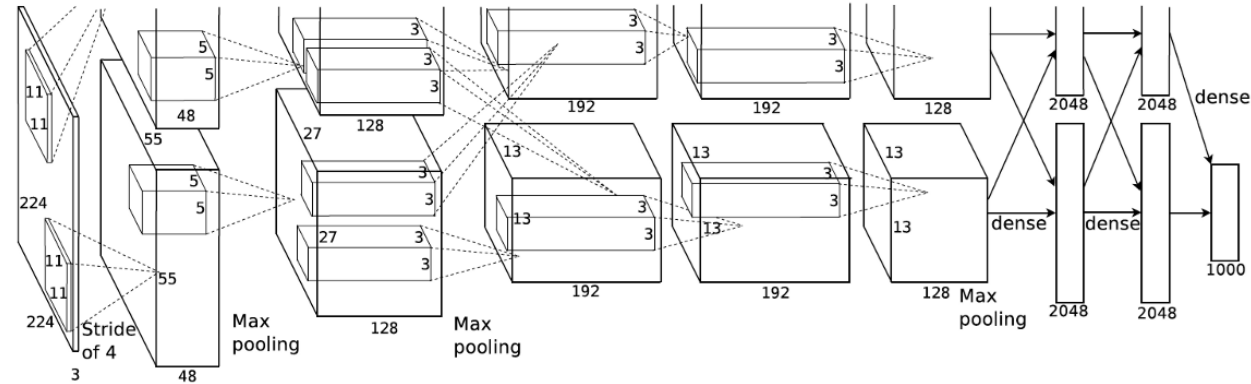
- Initializing a network with transferred features almost always gives better generalization

Summary And Insights:

- Pre-Training is important
 - With a related data set, synthetic data set.
- Find proxy/related tasks and learn features.
 - Unsupervised (eg. Autoencoder)
 - Word2Vec
 - Siamese
- Finetune/adapt/transfer learn
 - Work with smaller number of examples

Blank Slide

AlexNet



Full (simplified) AlexNet architecture:

[227x227x3] INPUT

[55x55x96] CONV1: 96 11x11 filters at stride 4, pad 0

[27x27x96] MAX POOL1: 3x3 filters at stride 2

[27x27x96] NORM1: Normalization layer

[27x27x256] CONV2: 256 5x5 filters at stride 1, pad 2

[13x13x256] MAX POOL2: 3x3 filters at stride 2

[13x13x256] NORM2: Normalization layer

[13x13x384] CONV3: 384 3x3 filters at stride 1, pad 1

[13x13x384] CONV4: 384 3x3 filters at stride 1, pad 1

[13x13x256] CONV5: 256 3x3 filters at stride 1, pad 1

[6x6x256] MAX POOL3: 3x3 filters at stride 2

[4096] FC6: 4096 neurons

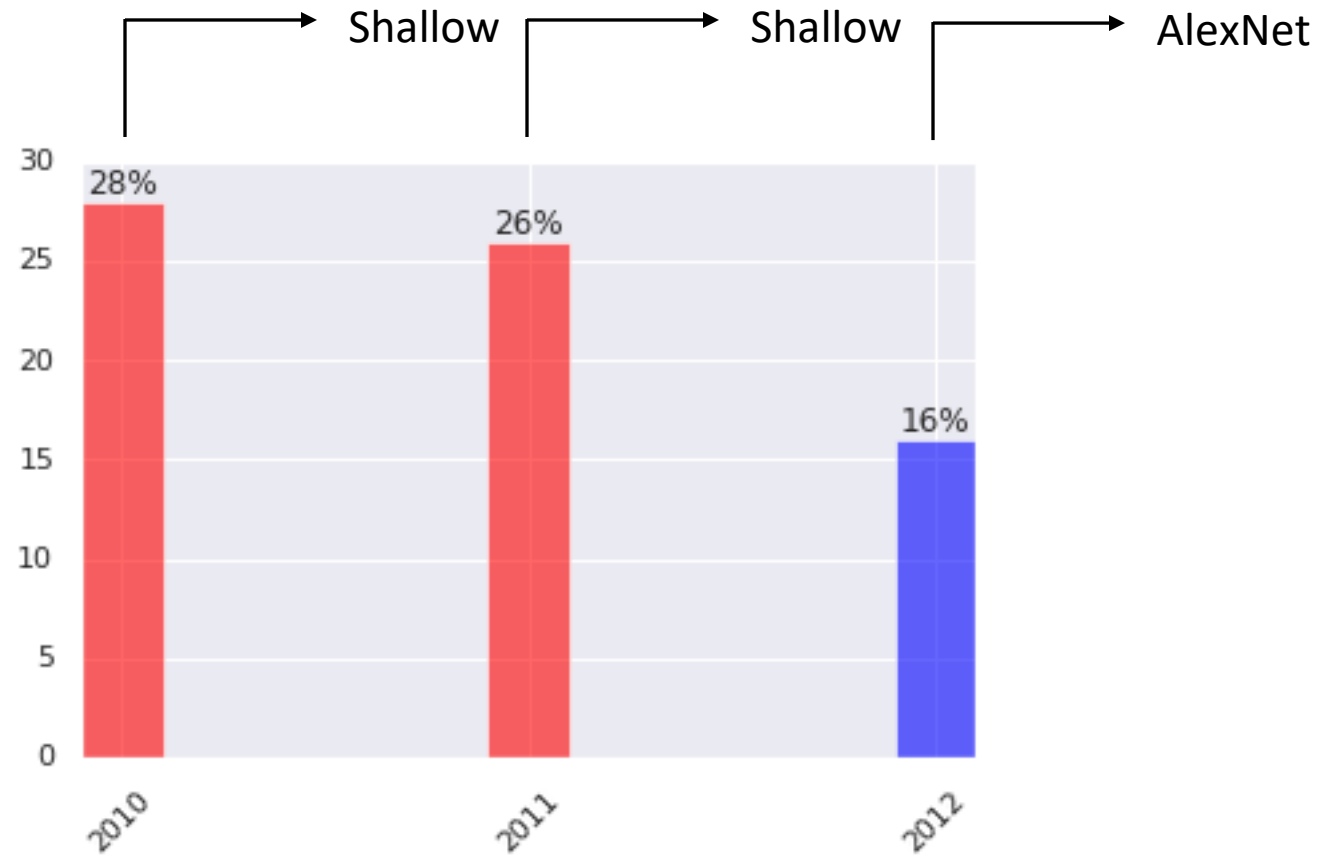
[4096] FC7: 4096 neurons

[1000] FC8: 1000 neurons (class scores)

PyTorch Class for AlexNet:

```
1 import torchvision
2
3 alexnet_model = torchvision.models.alexnet(pretrained = True)
```

AlexNet: Performance on ImageNet



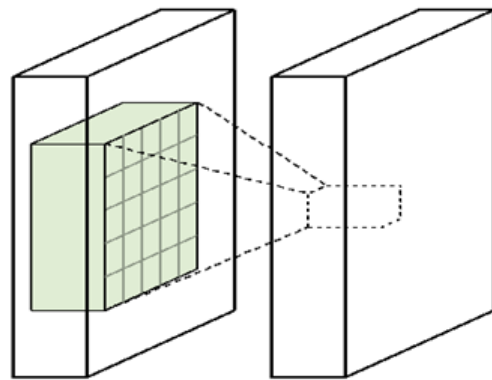
Improvements over AlexNet - I

- Smaller receptive field throughout the network
 - AlexNet used $k = 11 \times 11$ and $s = 4$
 - VGGNet used $k = 3 \times 3$ and $s = 1$

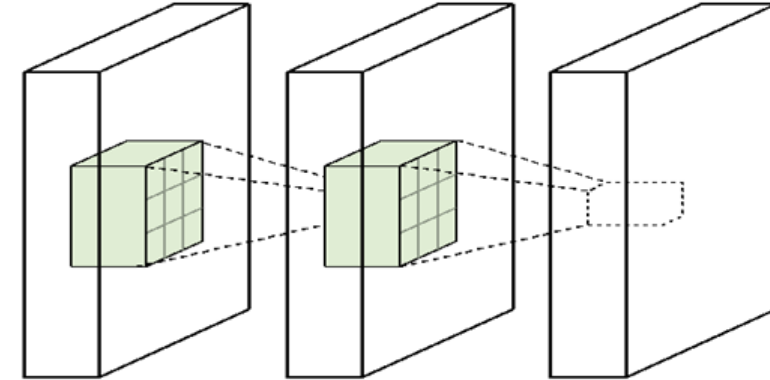
Intuition: A stack of **two** 3×3 convolutional layer is equivalent to a 5×5 convolution layer

- More non-linearity
- Less number of parameters

Design Guidelines



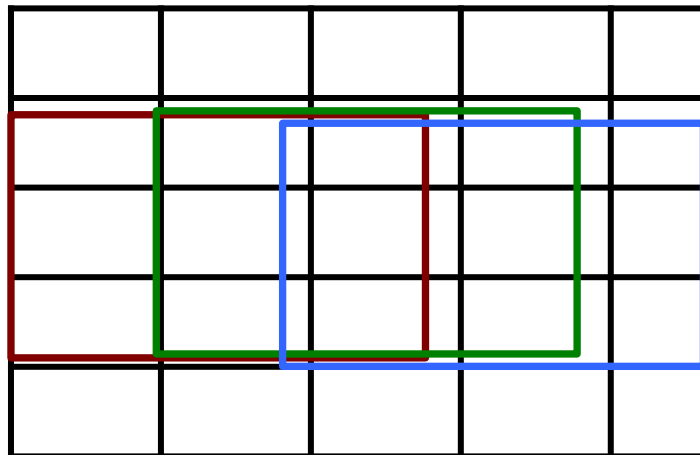
5 × 5 filters
+ ReLU



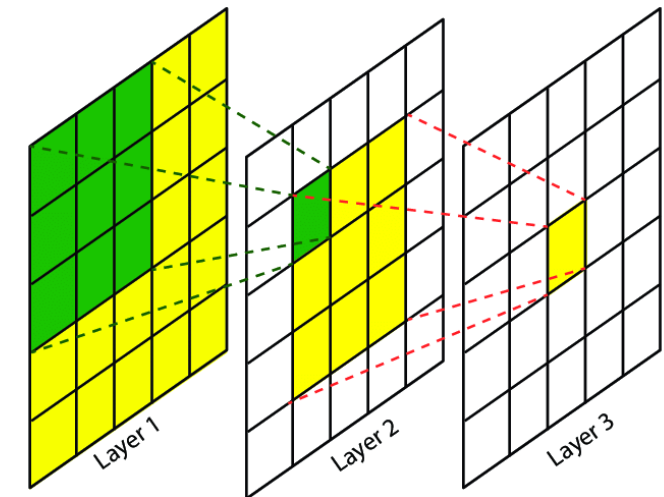
3 × 3 filters
+ ReLU

3 × 3 filters
+ ReLU

$5 \times 5 C > 2 \times 3 \times 3 C$



1. Less Parameters; Faster
2. Same Receptive Field
3. More nonlinearities (2 ReLU)



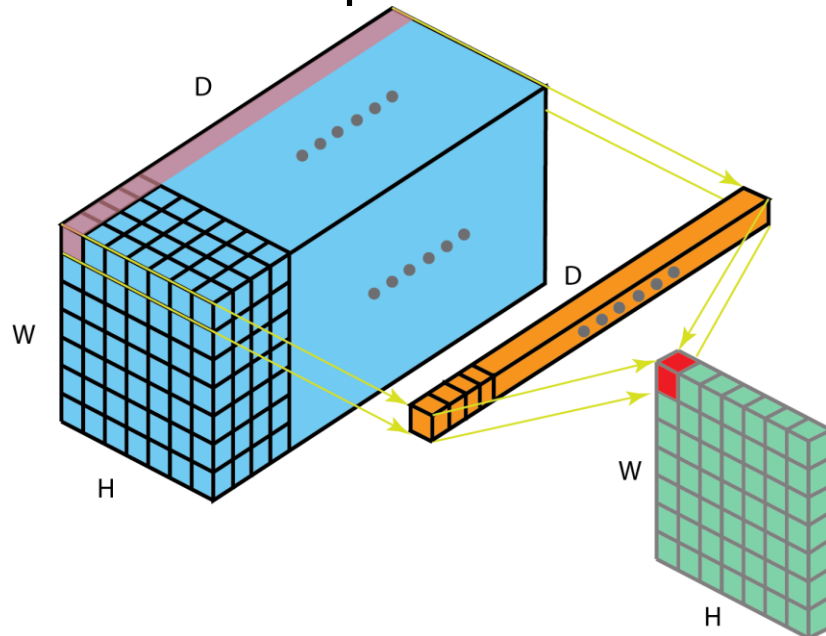
Blank Slide

Blank Slide

Improvements over AlexNet - II

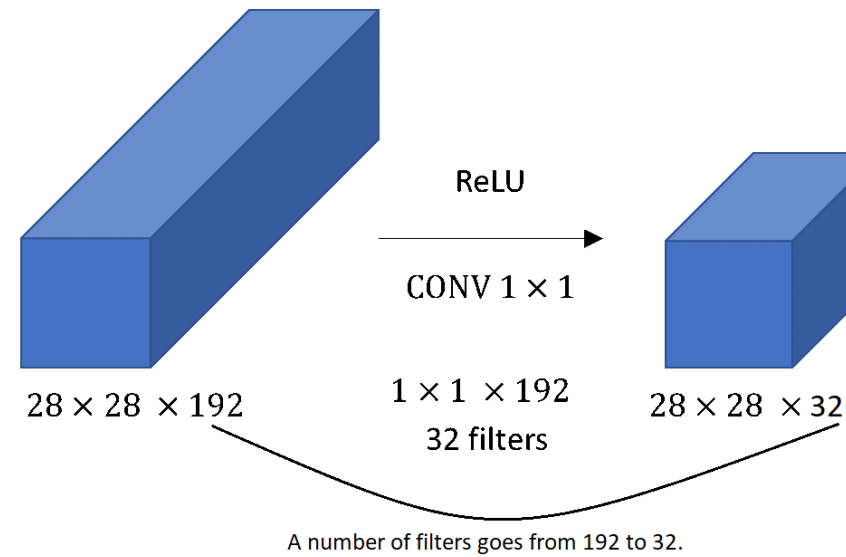
- 1×1 convolution

Increases the non-linearity without affecting the receptive field



Usage:

- Dimensionality reduction
- Building deeper network w/ large increase in parameters
- Increased non-linearity



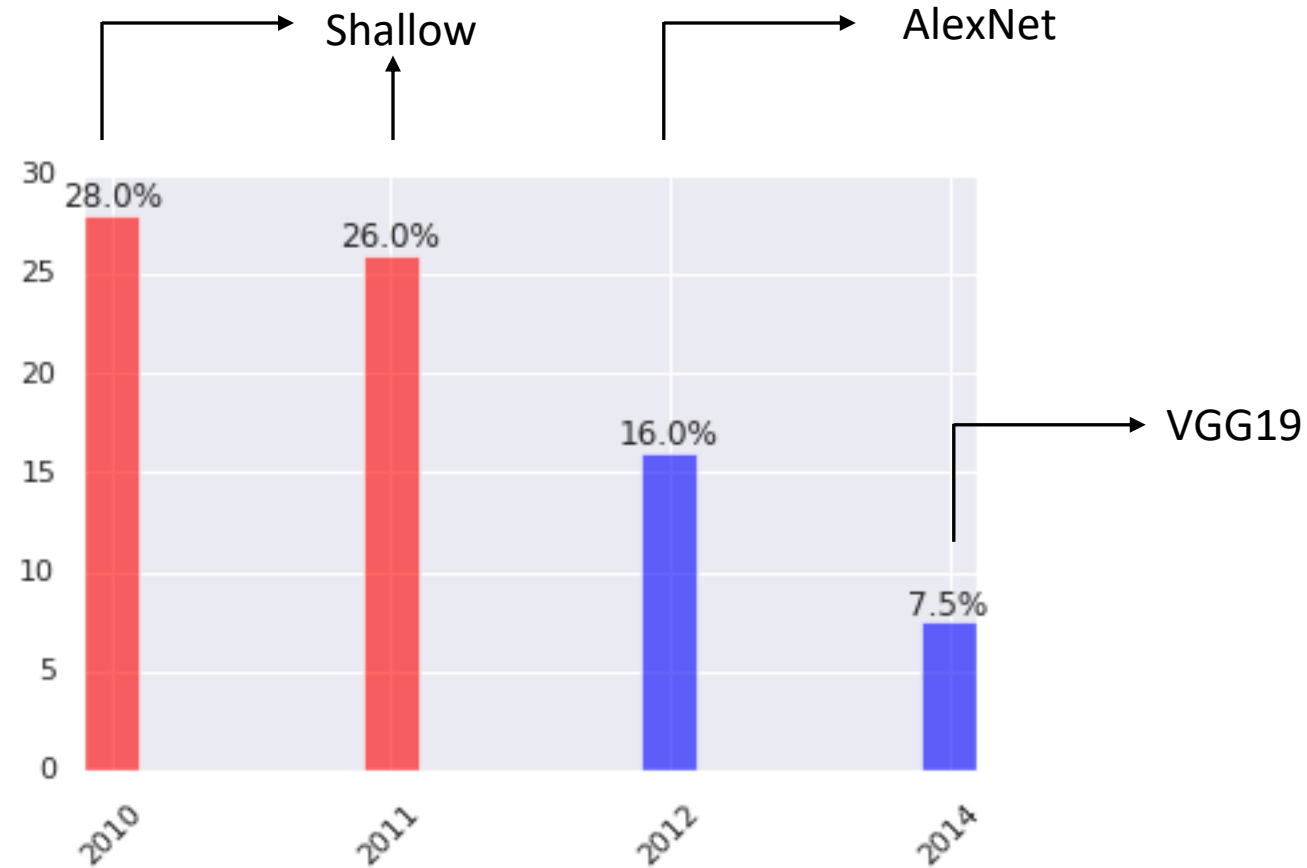
Blank Slide

Improvements over AlexNet - III

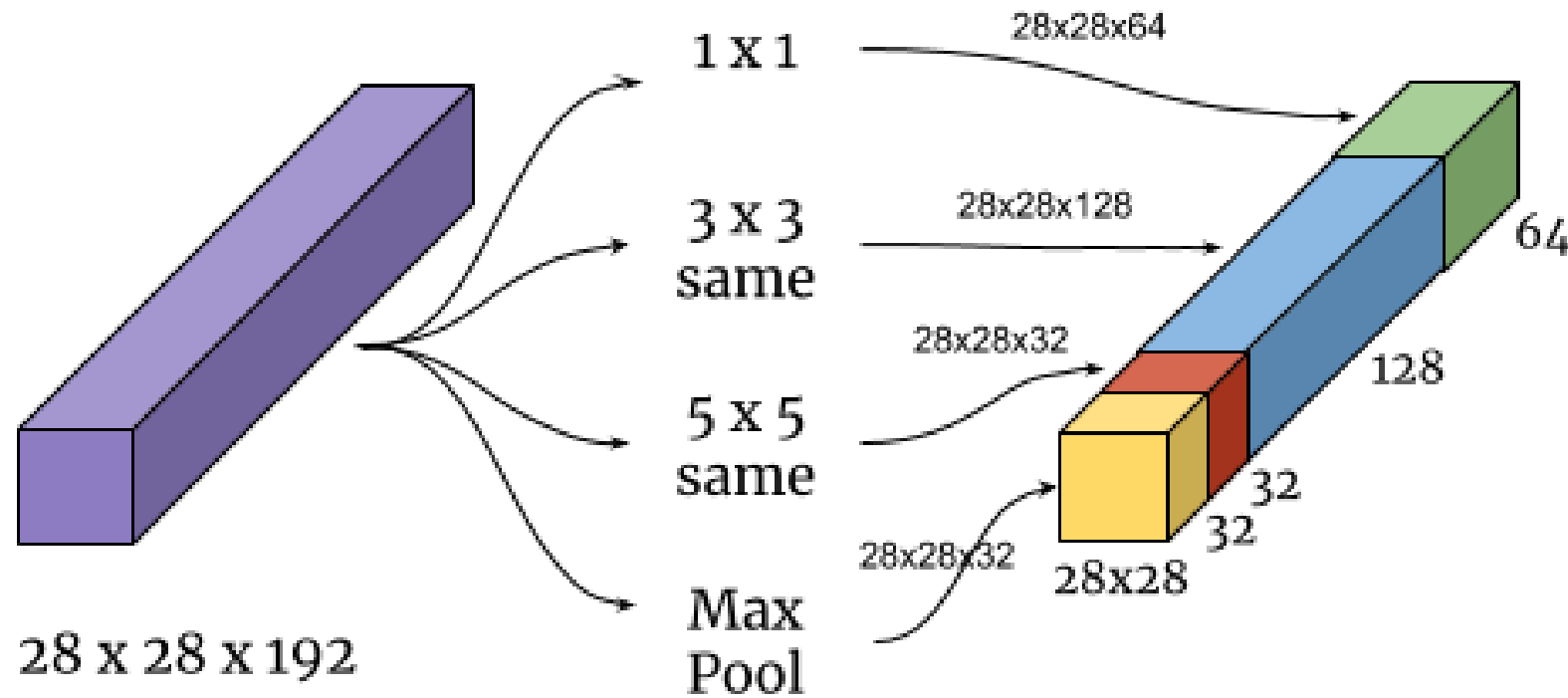
- Deeper Networks
 - VGG-A, VGG-A-LRN (11 layers)
 - 133M parameters
 - VGG-B (13 layers)
 - 133M parameters
 - VGG-C, VGG-D (16 layers)
 - 134M and 138M parameters
 - VGG-E (19 layers)
 - 144M parameters

```
1 import torchvision
2
3 vggnet_11_model = torchvision.models.vgg11(pretrained = True)
4 vggnet_13_model = torchvision.models.vgg13(pretrained = True)
5 vggnet_16_model = torchvision.models.vgg16(pretrained = True)
6 vggnet_19_model = torchvision.models.vgg19(pretrained = True)
7
```

VGGNet: Performance on ImageNet

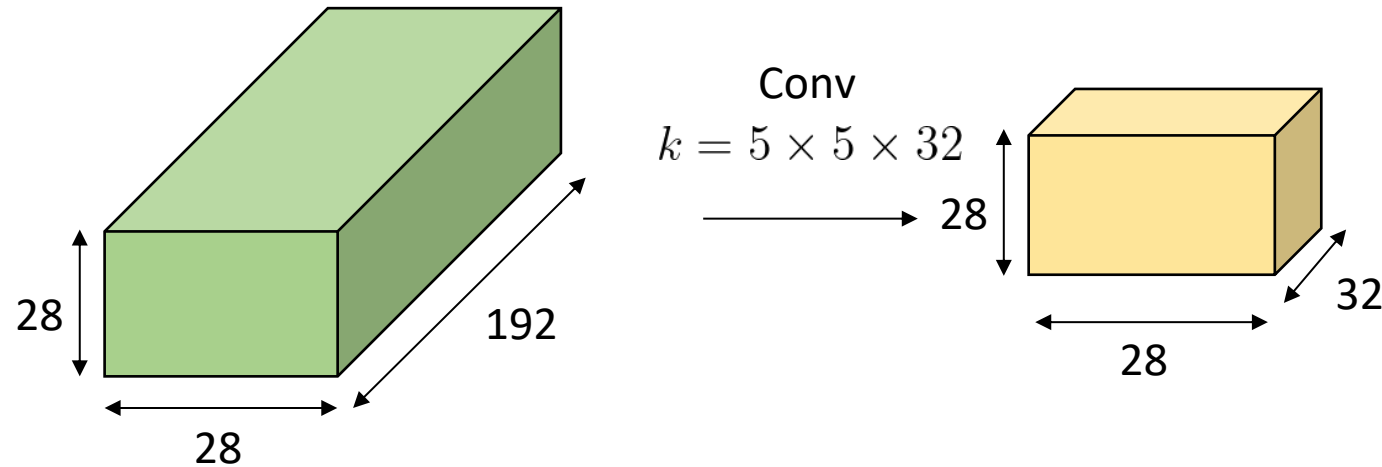


GoogLeNet



This would take a large number of computations! Can we reduce it?

Motivation of using a 1x1 Convolutional Layer:

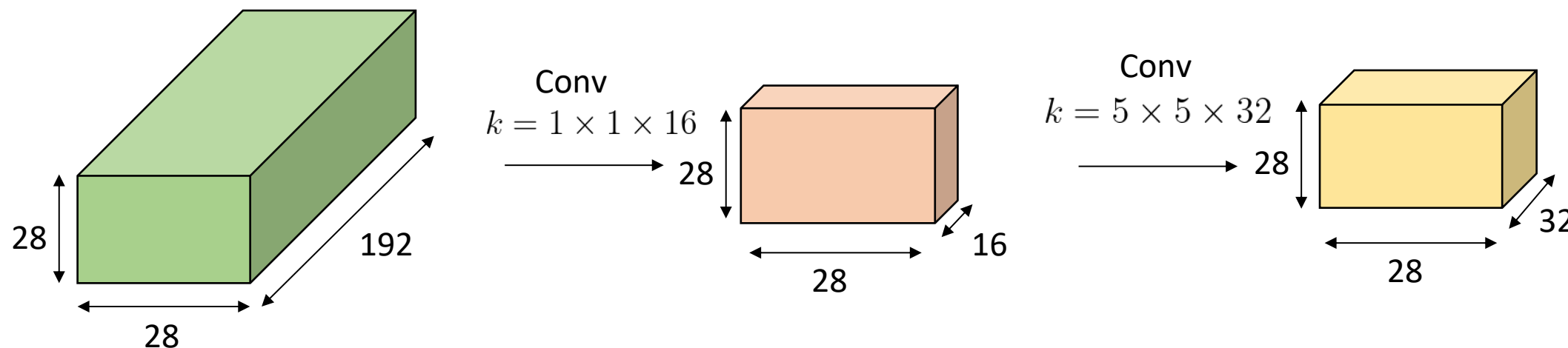


Total number of operations:

$$(28 \times 28 \times 32) \times (5 \times 5 \times 192) = 120\text{M}$$

GoogLeNet

Motivation:



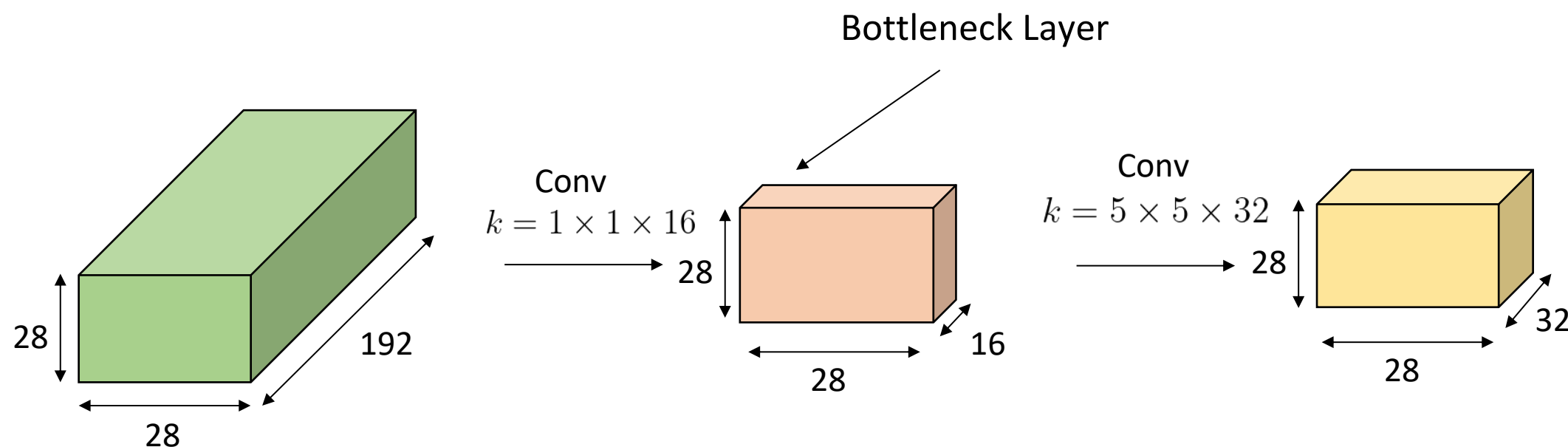
Total number of operations:

$$(28 \times 28 \times 16) \times (1 \times 1 \times 192) + (28 \times 28 \times 32) \times (5 \times 5 \times 16) = 12.4\text{M}$$

GoogLeNet



Motivation:

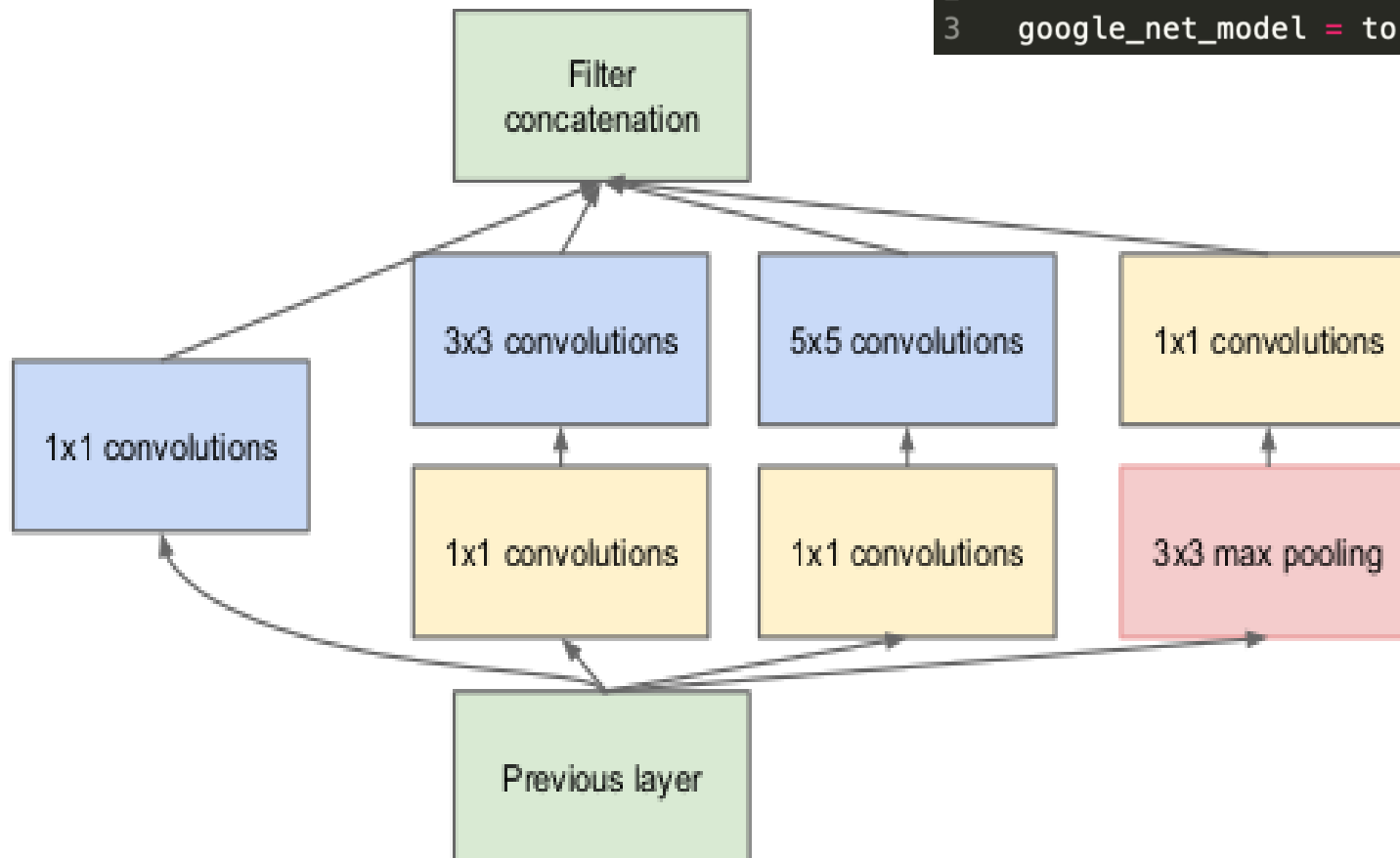


Total number of operations:

$$(28 \times 28 \times 16) \times (1 \times 1 \times 192) + (28 \times 28 \times 32) \times (5 \times 5 \times 16) = 12.4\text{M}$$

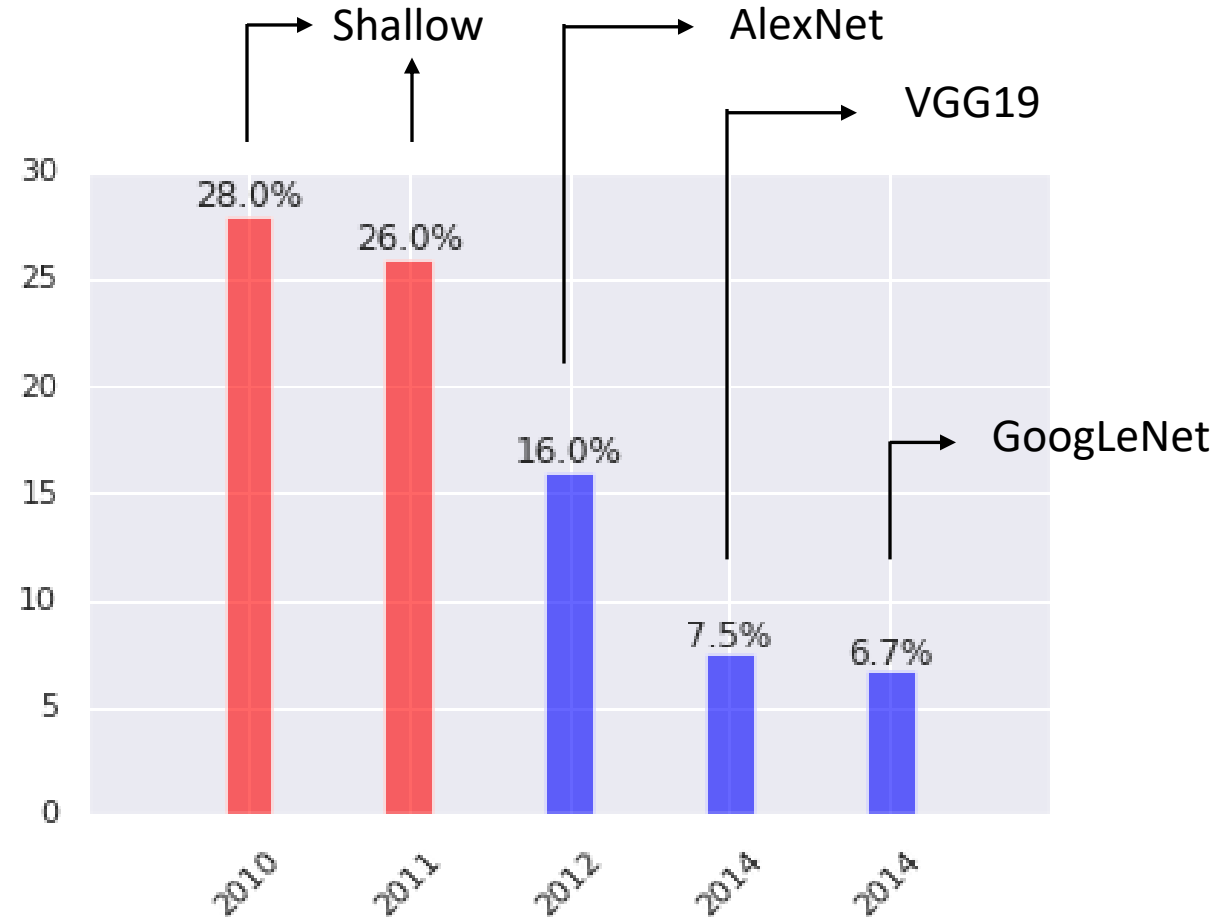
Blank Slide

The Inception Layer



```
1 import torchvision
2
3 google_net_model = torchvision.models.googlenet(pretrained = True)
```

GoogLeNet: Performance on ImageNet

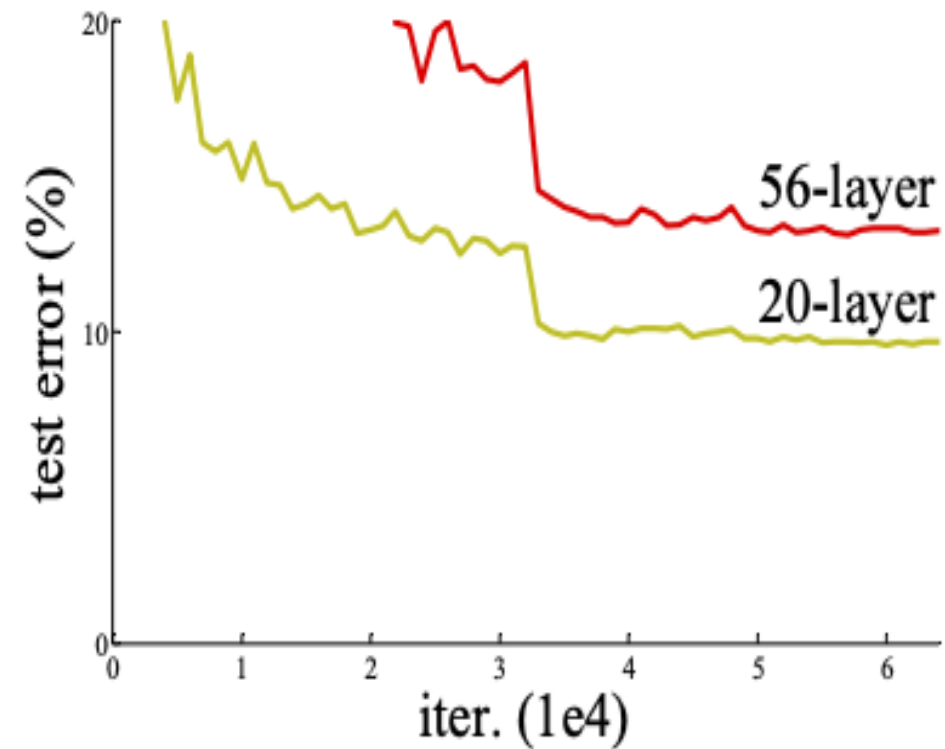
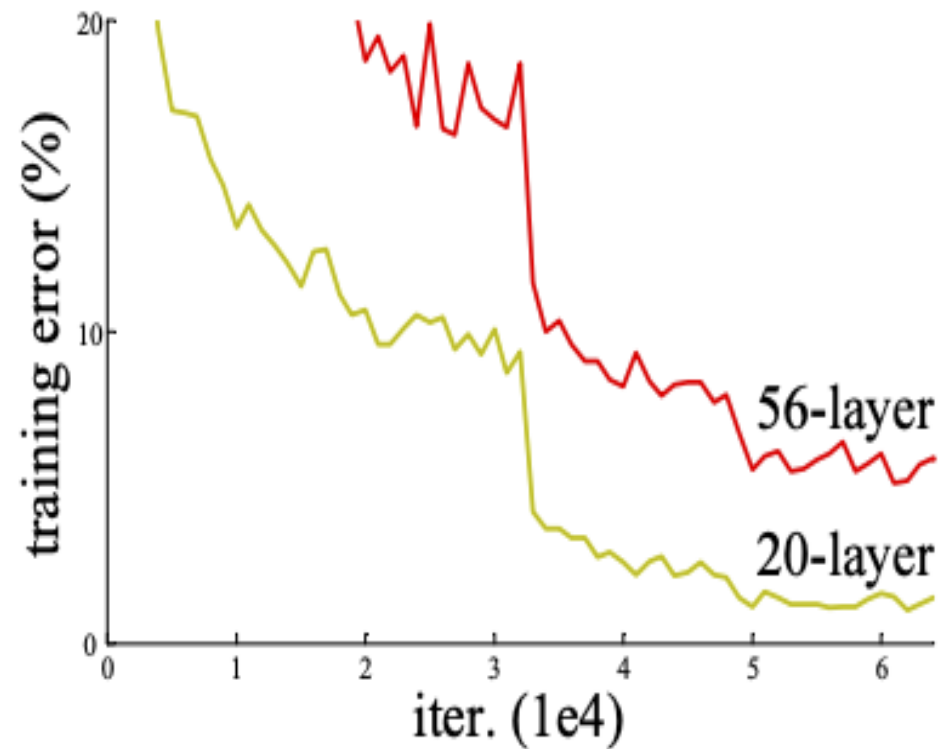


Does increasing the depth of a network always leads to better performance?

ResNets

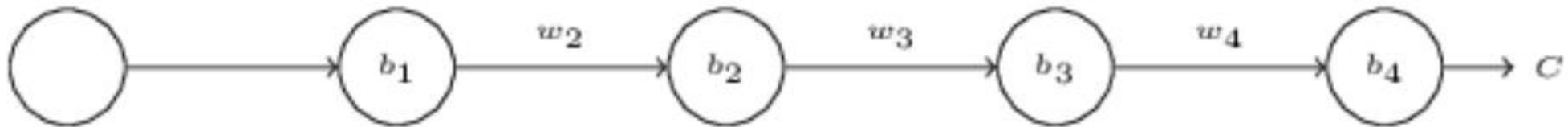
Does increasing the depth of a network always leads to better performance?

No!!



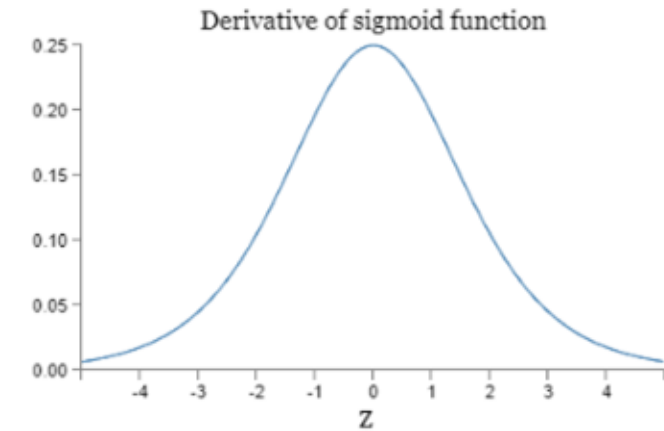
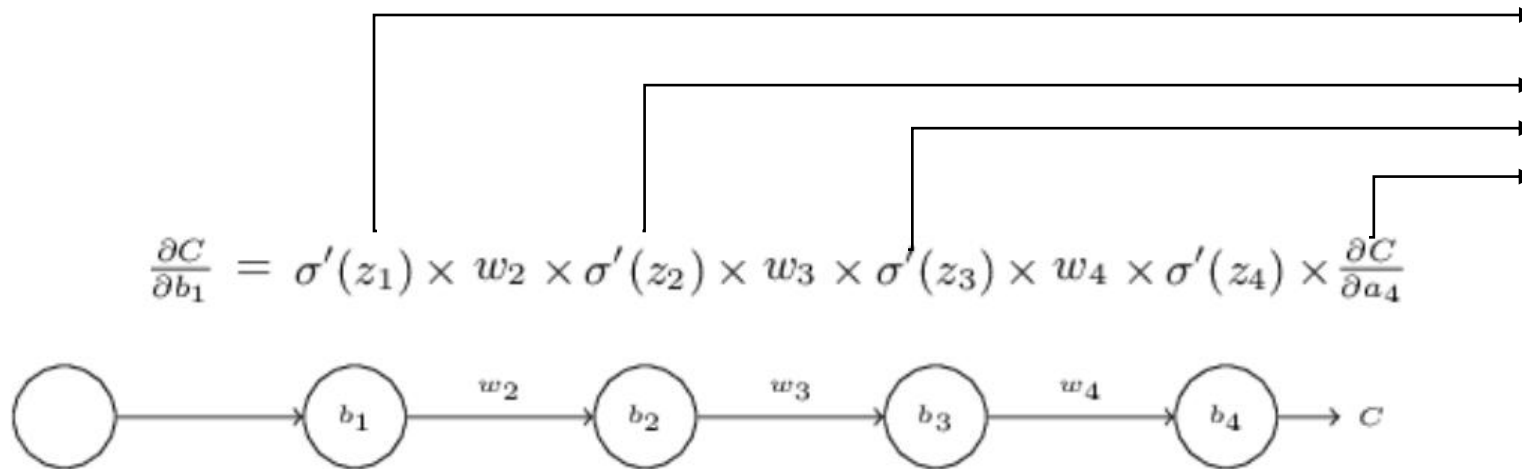
The Vanishing Gradient Problem:

$$\frac{\partial C}{\partial b_1} = \sigma'(z_1) \times w_2 \times \sigma'(z_2) \times w_3 \times \sigma'(z_3) \times w_4 \times \sigma'(z_4) \times \frac{\partial C}{\partial a_4}$$



ResNets

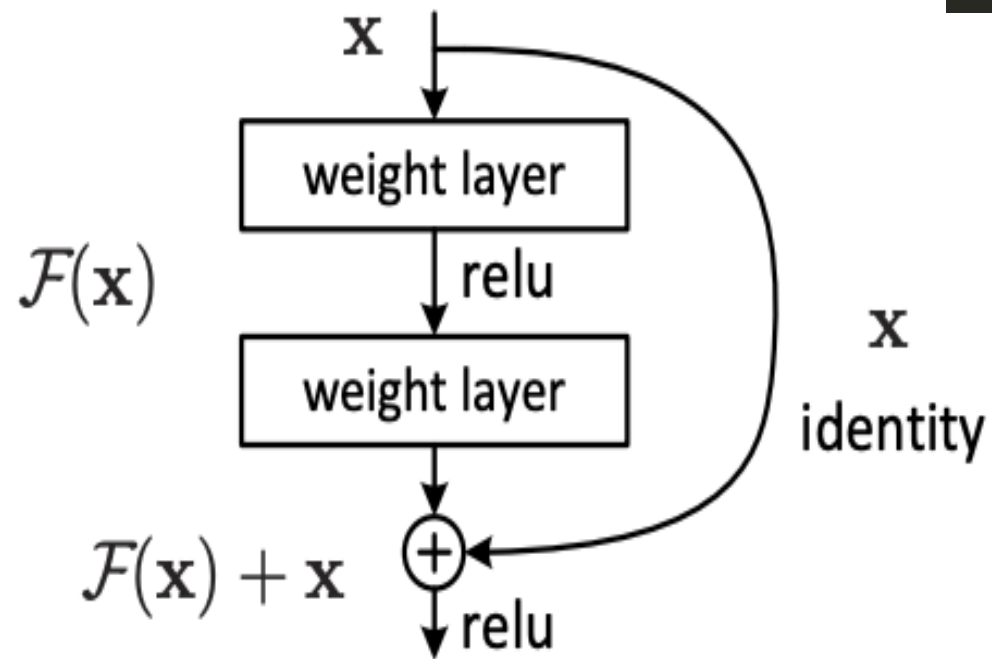
The Vanishing Gradient Problem:



ResNets

Solution: Skip Connection

```
1 import torchvision
2
3 resnet_18_model = torchvision.models.resnet18(pretrained = True)
```

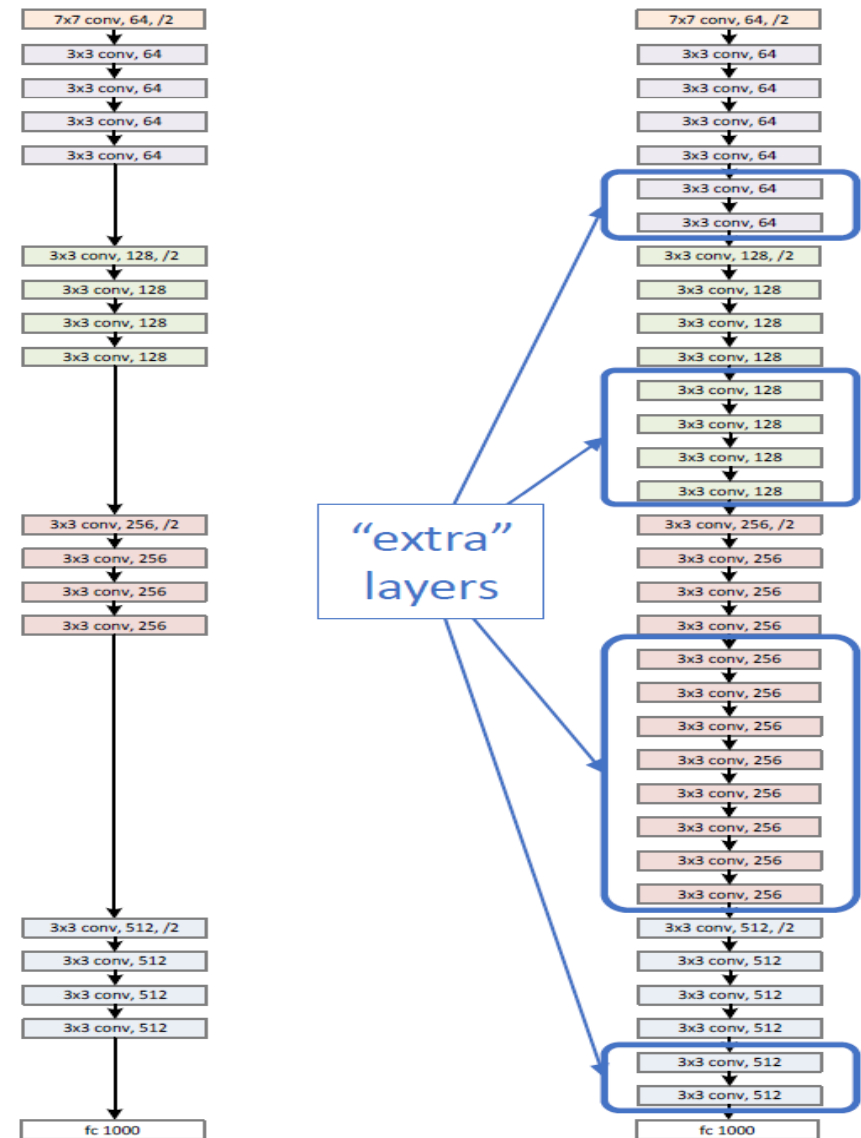


Blank Slide

Blank Slide

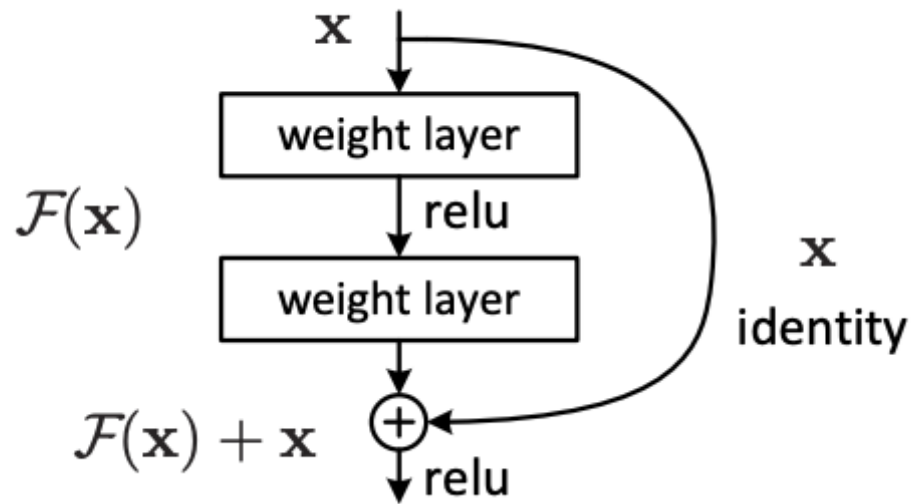
Simple Argument

- Naïve solution
 - If extra layers are an **identity** mapping, then training errors do not increase



Blank Slide

Gradient through Skip Connection

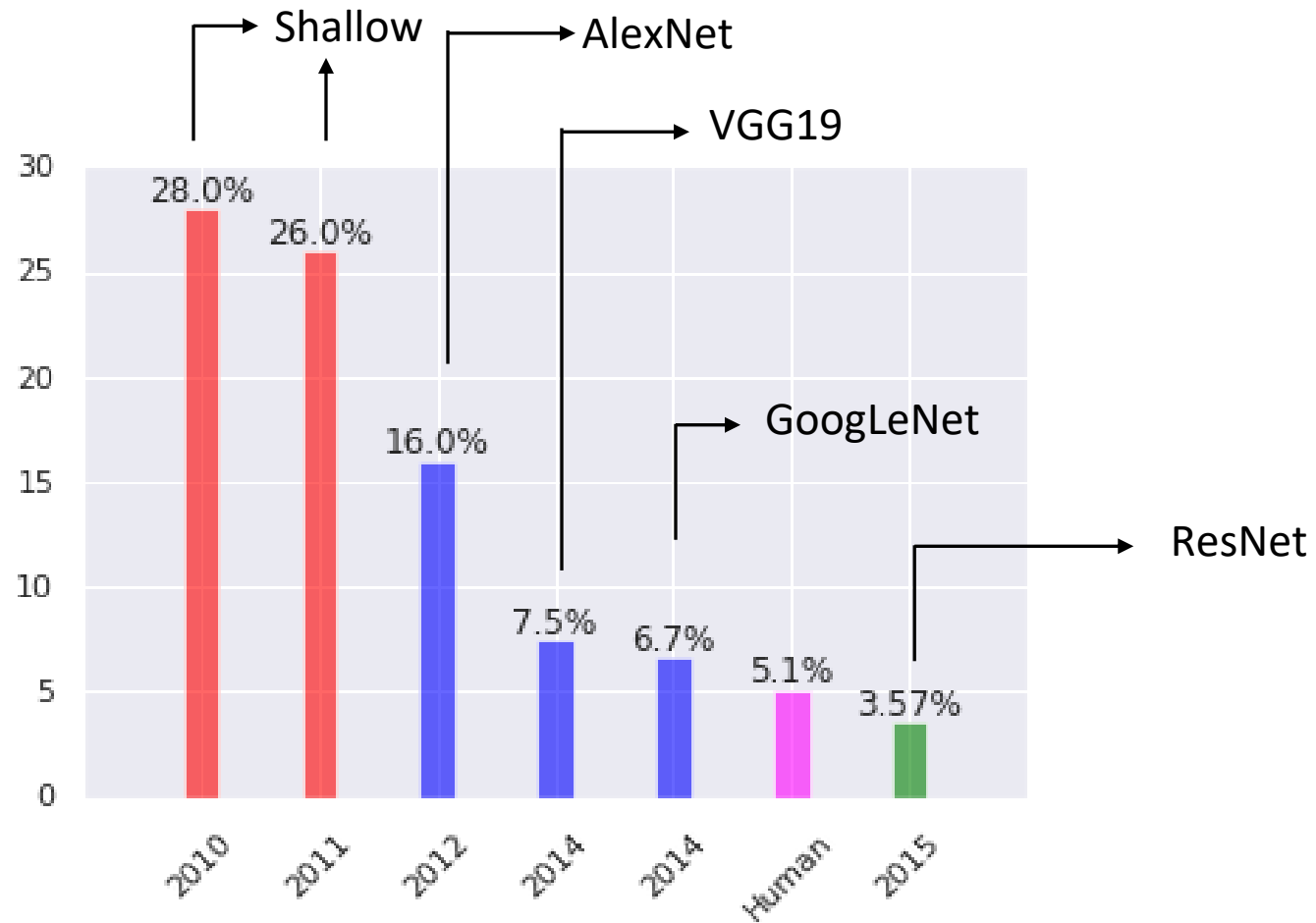


$$\begin{aligned} y &= x + \mathcal{F}(x) \\ \frac{\delta L}{\delta x} &= \frac{\delta L}{\delta y} \frac{\delta y}{\delta x} \\ &= \frac{\delta L}{\delta y} \left(1 + \frac{\delta \mathcal{F}(x)}{\delta x} \right) \end{aligned}$$

Gradient from later layer directly passed to the earlier layers!

Blank Slide

ResNets: Performance on ImageNet



Which network to choose when?

Which Architecture is the Best?

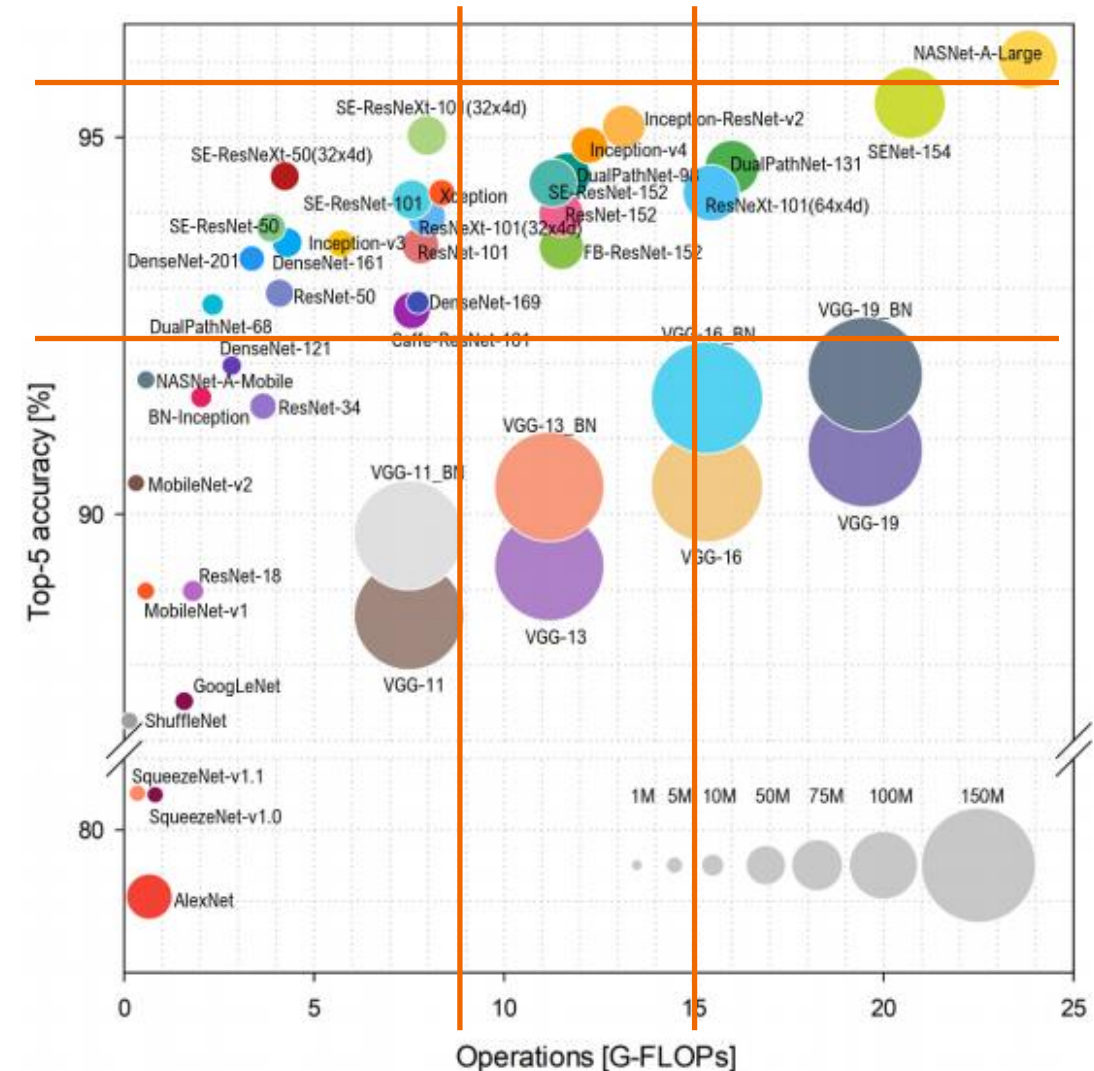
- ImageNet is to benchmark deep neural networks for image classification task
- All are designed to better accuracy
- Should we choose a network solely based on their performance?
- What are your production constraints?
- How to quantify them into the reasonable metrics to evaluate a CNN?

Performance Indices

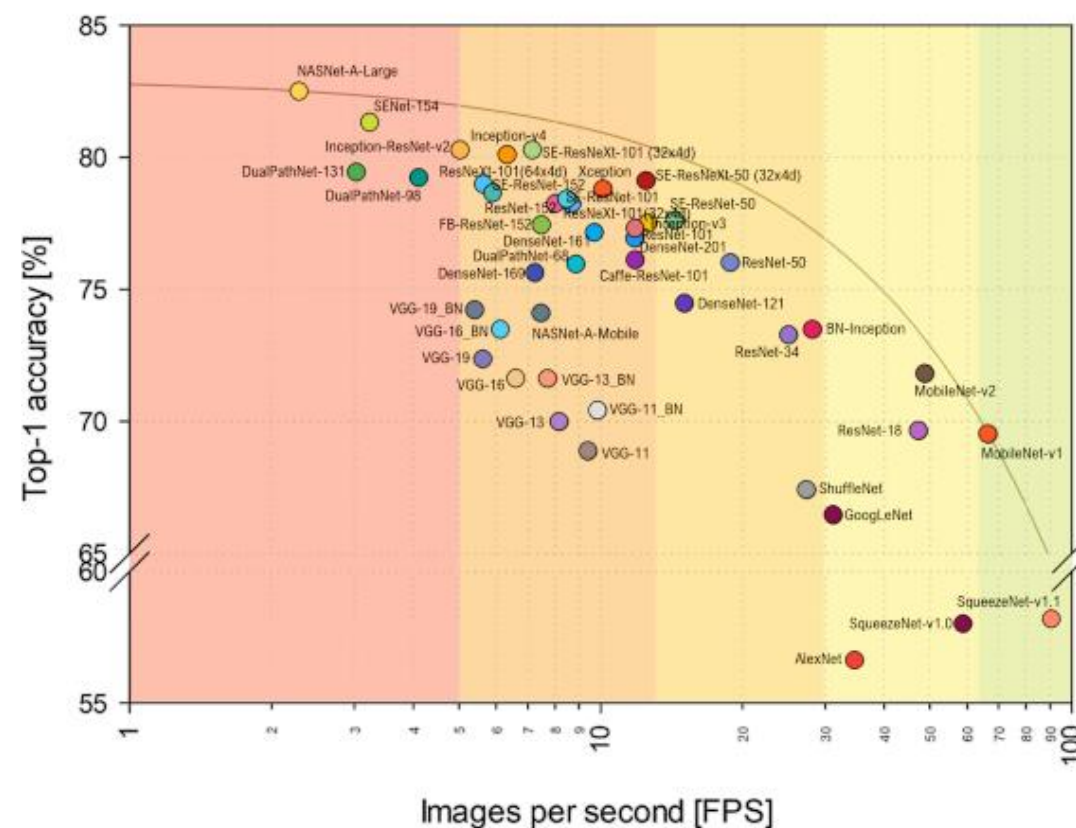
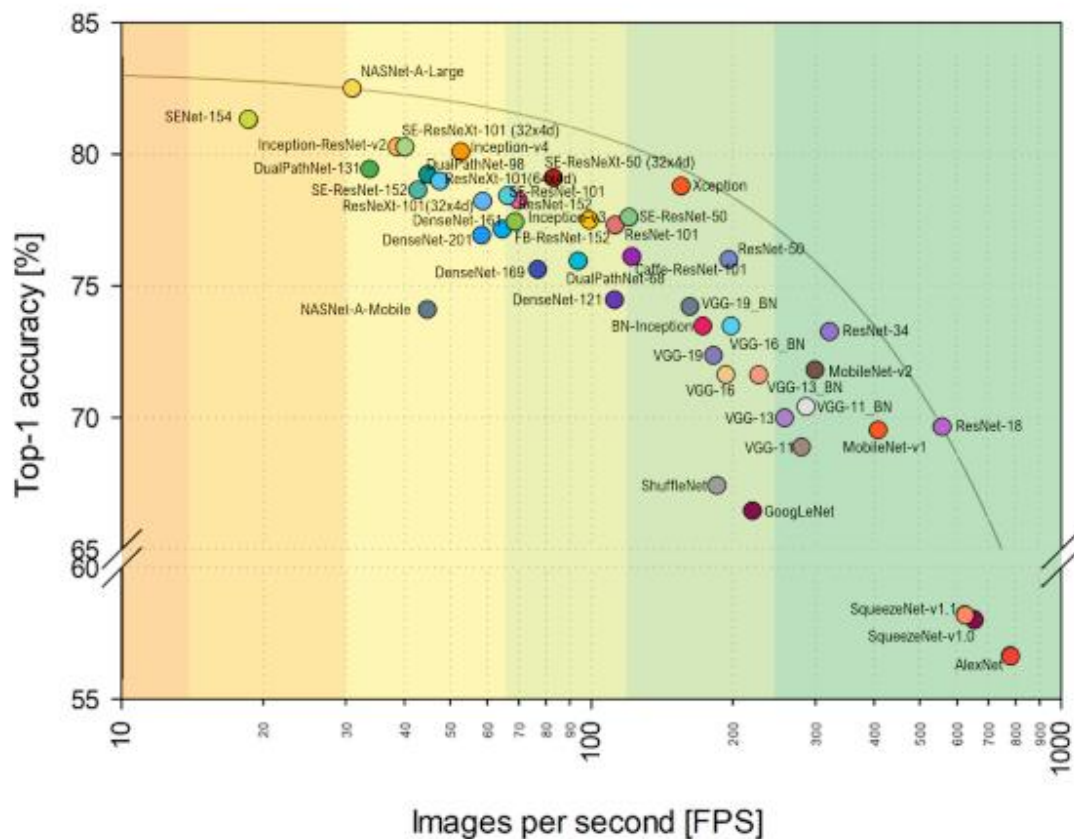
- Accuracy
- Model complexity
- Memory usage
- Computational complexity
- Inference time

Accuracy vs. Model Complexity vs. Computational Complexity

- Size of point denotes the Model complexity
- The band around 95% accuracy has varying complexity of 4-25 G-FLOPs
- The band between 10-15 G-FLOPs have high variance in both Model Complexity (size of the point) and accuracy
- Recognition accuracy is not only dependent on the model or computational complexity

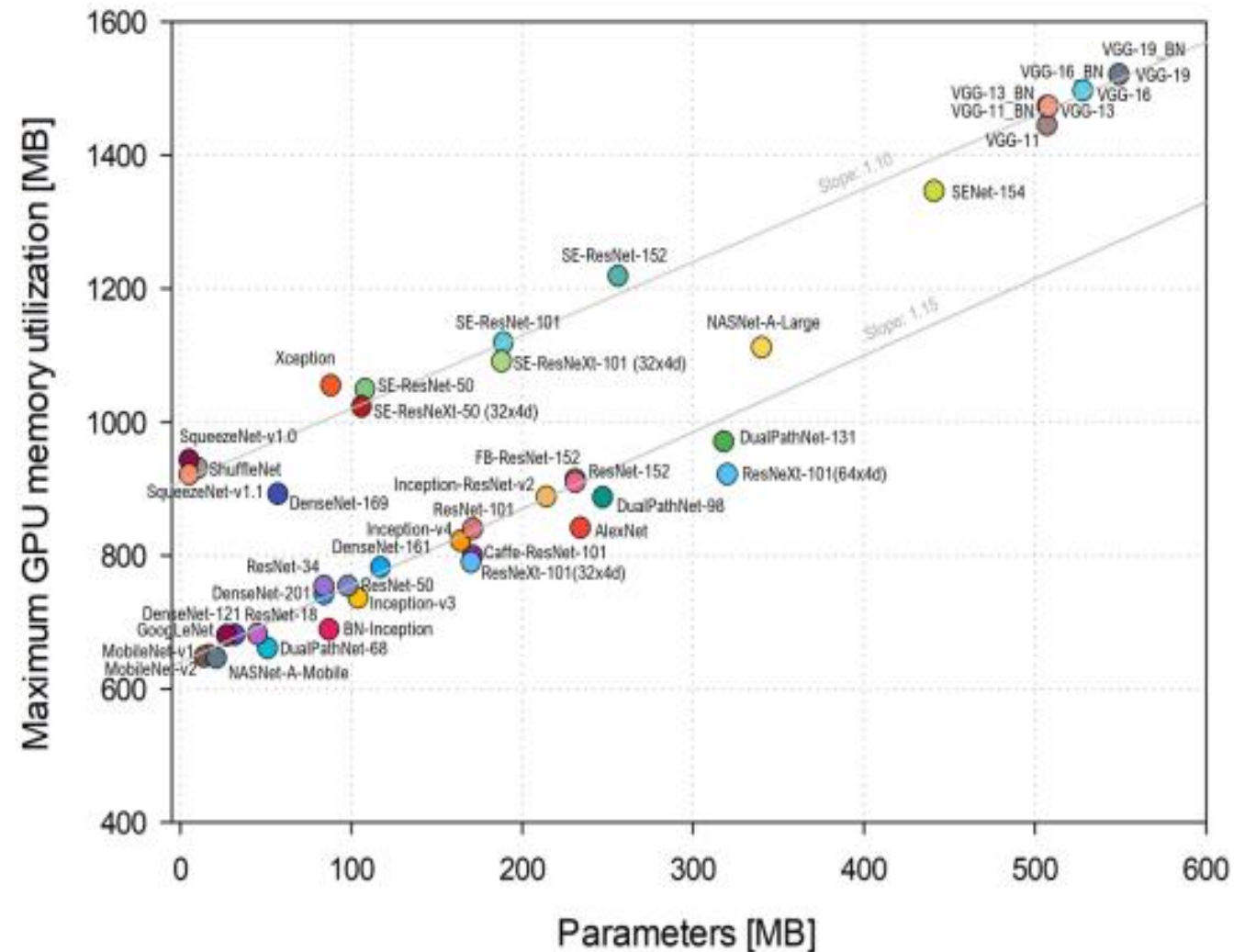


Accuracy vs. Model Complexity vs. Computation... Complexity



Model Complexity vs. Memory Usage

- Follows nearly a linear relationship
- Higher the complexity, higher memory it takes



Thanks!!

Questions?